

Excellent choice! Here's a complete **project preparation guide/story for MessFinder** — like a master document that teaches someone the **what, why, how, and everything in between**.

MessFinder – Complete Project Story & Notes for Preparation

For full understanding and strong viva performance

1. Project Overview / Introduction

MessFinder is a web-based platform designed to help students, job seekers, and working professionals easily find PG accommodations and rooms for rent. The application also allows property owners to post room listings and communicate directly with potential tenants.

This project was developed as a **Final Year Project** by students from **Bankura Sammilani College**, Computer Science Department. It addresses the real-life problem of **finding verified, affordable accommodation in an organized, secure, and accessible way**.

2. Project Motivation

In small towns, urban fringes, and even cities, **finding rental rooms or PGs** is often done via:

- Posters stuck on walls
- Word of mouth
- Unverified Facebook posts
- Local brokers (some are unreliable or expensive)

There is **no centralized, trusted platform** to find such places — especially for students moving to a new city.

Why MessFinder?

- To simplify the process of finding safe accommodation
- To create **trust** between room seekers and property owners
- To offer a **digital alternative** to messy and unorganized methods

- To give **PG owners visibility** without paying commission to middlemen
-

3. How the Application is Built

MessFinder follows a structured development process using **Agile** methodology (incremental development):

Development Steps:

1. Requirement Gathering:

- Identifying user types: `room seekers` and `owners`
- Understanding the challenges they face

2. System Planning:

- Designing data structure (rooms, users, messages)
- Planning components like search, dashboard, contact

3. UI/UX Design:

- Using Tailwind CSS to create a clean and responsive interface

4. Coding & Integration:

- React + Firebase-based frontend & backend development
- Adding role-based routing, forms, and filters

5. Testing:

- Manual and peer testing with dummy data
- Fixing edge case bugs

6. Deployment:

- Hosting frontend on **Vercel**
- Using Firebase for DB, Auth, and Hosting

7. Maintenance & Improvement:

- Adding new features like chat, security rules, etc.
-

4. Tech Stack Used

Component	Tool/Platform	Why We Used It
Frontend	React.js	Component-based, reusable, fast
Styling	Tailwind CSS	Utility-first, mobile-first, responsive
Routing	React Router	For page navigation
Backend	Firebase	No server needed; handles auth, DB, hosting
Database	Firestore (NoSQL)	Real-time, scalable, document-based
Storage	Firebase Storage	To store images securely
Authentication	Firebase Auth	Secure, fast login (email/Google)
Hosting	Vercel + Firebase	Easy, free, and fast deployments

5. User Roles and Flow

Room Seekers (Users)

- Can browse/search PGs using filters (location, gender, etc.)
- Can save favorites (if logged in)
- Can contact owners through messaging
- Can view PG details with images and contact options

Property Owners

- Register/Login as "Owner"
- Post their PG/room with images, location, details
- Edit or delete their listings
- Respond to messages from seekers

Common Flow:

1. User visits home page
2. Enters location/type to search rooms
3. Views filtered results
4. Logs in to message or bookmark a room
5. Owners log in to add/edit listings

6. Key Features

- **Search with Filters** (e.g., city, mess type, gender-based PGs)
 - **Post Room with Photos and Details**
 - **Real-time Messaging** with owners
 - **Login with Firebase Auth** (secure and simple)
 - **Mobile Responsive Design**
 - **Role-based access control**
 - **Location tagging for listings**
 - **Clean, fast-loading UI**
-

7. System Structure Overview

Firestore DB:

```
/users/{userId}
/rooms/{roomId}
/messages/{chatId}/chats/{messageId}
```

Pages:

- `/` → Home & Search
 - `/details/:id` → Room Details
 - `/submit` → Owner Submit Room
 - `/dashboard` → Owner/User Control Panel
 - `/login` → Firebase login
 - `/profile/:ownerId` → Public Owner Profile
-

8. Security Measures

- Firebase Authentication for verified login
- Firestore Rules:
 - Only owners can write to their own rooms
 - Only users with roles can access certain routes
- HTTPS ensures all communication is encrypted

- Secure Cloud Storage access with token-based upload
-

9. Feasibility & Practicality

- Works on any smartphone or desktop browser
 - Hosting is free (Vercel + Firebase)
 - Fast load times and low maintenance
 - Scalable due to cloud-based architecture
-

10. Social Impact

- Helps students and migrants find safe accommodation
 - Supports local PG owners in small towns
 - Promotes digital literacy and modern platforms
 - Reduces dependency on brokers or shady deals
-

11. Future Plans

- Add **Map-based Search** with geolocation
 - **Online Payments** for booking
 - **Push Notifications**
 - **Review & Rating System**
 - **Accessibility Options** (for differently-abled)
 - **Admin Dashboard** to manage reported content
-

12. Project Learning Outcomes

- Building a **full-stack serverless application**
 - Understanding **role-based access** and secure data flows
 - Learning **modular frontend architecture**
 - Team collaboration using GitHub, Google Docs
 - Managing time, debugging, and testing in real scenarios
-

13. Final Tip for Viva

- Be confident. Speak in simple language.
- Know your feature list, tech stack, and what each team member did.
- If you're unsure, say:

“That’s something we considered but haven’t implemented yet.”
